Arrays & Linked List

CSE 4303: Data Structures
Lecture 2.1

Asaduzzaman Herok
Department of CSE
Islamic University of Technology

November 17, 2025

🤔 Arrays

Problem Scenario

There are 20 students in a class, we have write a program that reads and prints the marks of all the students. How can we do it?

Using Primitive types

- 👉 Need 20 integer variables for storing and printing.
- 👉 Need 20 statements for input.
- 👉 Need 20 more statements for output.

A Better Way

Use arrays of integers. Since working with homogeneous data.

⚡ Complexity of Array Operations.

Operations	Front/1st	Arbitrary position	Back/ n^{th}
Find	$\Theta(1)$	$\Theta(n)$*, $\Theta(log(n))^{\dagger}$	$\Theta(n)$
Insert	$\Theta(n)$	$\Theta(n)$	$\Theta(1)^{\ddagger}$, $\Theta(n)$ §
Erase	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$

Table: Time complexity of Array Operations

*Finding value in Unsorted Array
†Finding value in Sorted arrays
‡Array is not full
§Array is full

Problem Scenario

The Music Player Playlist

Develop the playlist module of a music player application (like Spotify). A playlist contains a sequence of songs, and the user must be able to modify the playlist while a song is playing.

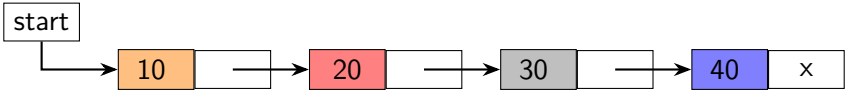
Requirments

- 👉 Adding a new song before or after the currently playing song
- 👉 Removing the currently playing song
- 👉 Instantly moving to the next or previous song
- 👉 Reordering songs dynamically (move up/down)
- 👉 Looping through songs (optional advanced feature)
- 👉 Handling large playlists (thousands of songs) without performance issues

Linked List

Linked List (Singly)

A linked list, in simple terms, is a linear collection of data elements. These data elements are called nodes.



Pros:

- 👉 Dynamic Size
- 👉 Efficient insertion and deletion
- 👉 No waste of memories.
- 👉 Easily used for complex data structures.

Cons:

- 👉 Random access is not possible.
- 👉 Extra memory for pointer variables.
- 👉 Searching is slow.
- 👉 Dynamic memory allocaition is slow.

Singly Linked List Operations (Traversing)

Traversing

Accessing the nodes of the list in order to perform some processing on them.

Pseudo Code

```
Step 1: [INITIALIZE] SET PTR = START
Step 2: Repeat Steps 3 and 4 while PTR != NULL
Step 3:         Apply Process to PTR -> DATA
Step 4:         SET PTR = PTR -> NEXT
        [END OF LOOP]
Step 5: EXIT
```

Singly Linked List Operations (Searching)

Searching:

Finding a particular element in the linked list. If it is present, the algorithm returns the address of the node that contains the value.

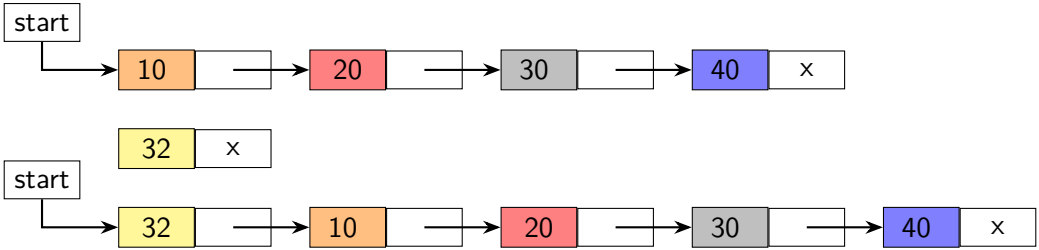
Pseudo Code

```
Step 1: [INITIALIZE] SET PTR = START
Step 2: Repeat Steps 3 and 4 while PTR != NULL
Step 3:     IF VAL == PTR -> DATA
                SET POS = PTR
                Go To Step 5
            ELSE
                SET PTR = PTR -> NEXT
            [END OF IF]
        [END OF LOOP]
Step 5: EXIT
```

Singly Linked List Operations (Insertion of New Node) I

Case 1

The new node is inserted at the beginning.



Singly Linked List Operations (Insertion of New Node) II

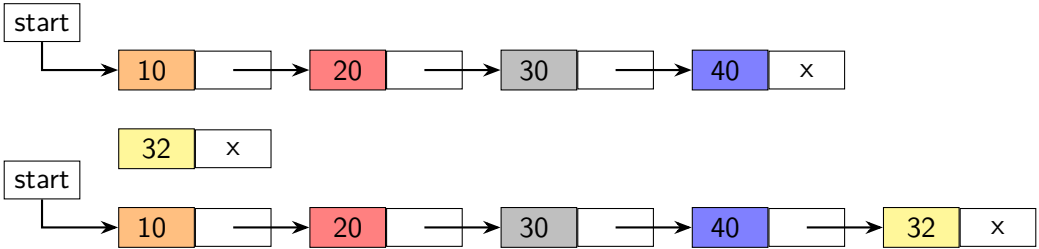
Pseudo Code

```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 7
        [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL -> NEXT
Step 4: SET NEW_NODE -> DATA = VAL
Step 5: SET NEW_NODE -> NEXT = START
Step 6: SET START = NEW_NODE
Step 7: EXIT
```

Singly Linked List Operations (Insertion of New Node) III

Case 2

The new node is inserted at the end.



Singly Linked List Operations (Insertion of New Node) IV

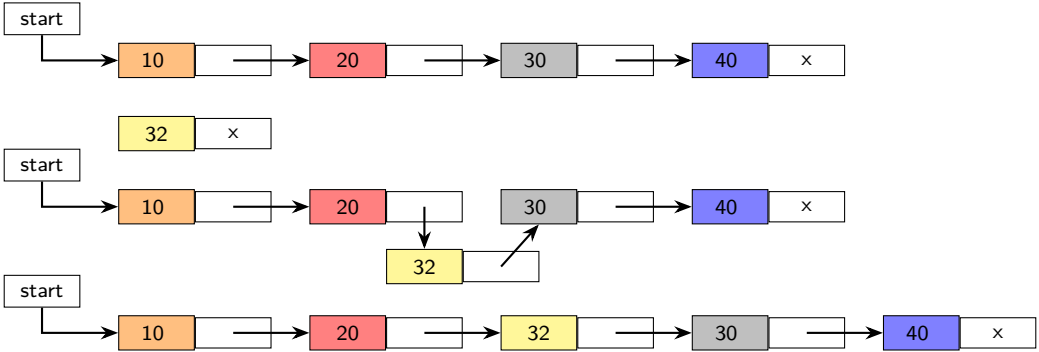
Pseudo Code

```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 10
        [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL - > NEXT
Step 4: SET NEW_NODE - > DATA = VAL
Step 5: SET NEW_NODE - > NEXT = NULL
Step 6: SET PTR = START
Step 7: Repeat Step 8 while PTR - > NEXT != NULL
Step 8:     SET PTR = PTR - > NEXT
        [END OF LOOP]
Step 9: SET PTR - > NEXT = NEW_NODE
Step 10: EXIT
```

Singly Linked List Operations (Insertion of New Node) V

Case 3

The new node is inserted after a given node.



Singly Linked List Operations (Insertion of New Node) VI

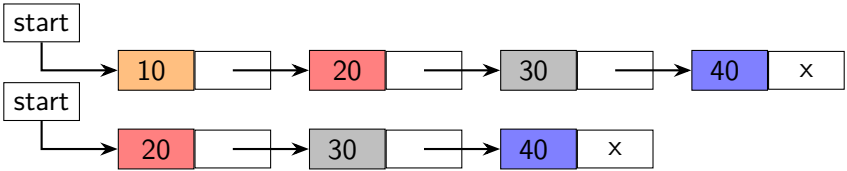
Pseudo Code

```
Step 1: IF AVAIL = NULL
            Write OVERFLOW
            Go to Step 12
        [END OF IF]
Step 2: SET NEW_NODE = AVAIL
Step 3: SET AVAIL = AVAIL - > NEXT
Step 4: SET NEW_NODE - > DATA = VAL
Step 5: SET PTR = START
Step 6: SET PREPTR = PTR
Step 7: Repeat Steps 8 and 9 while PREPTR - > DATA != NUM
Step 8:     SET PREPTR = PTR
Step 9:     SET PTR = PTR - > NEXT
        [END OF LOOP]
Step 1 : PREPTR - > NEXT = NEW_NODE
Step 11: SET NEW_NODE - > NEXT = PTR
Step 12: EXIT
```

Singly Linked List Operations (Deletion of a Node) I

Case 1

The first node is deleted.



Singly Linked List Operations (Deletion of a Node) II

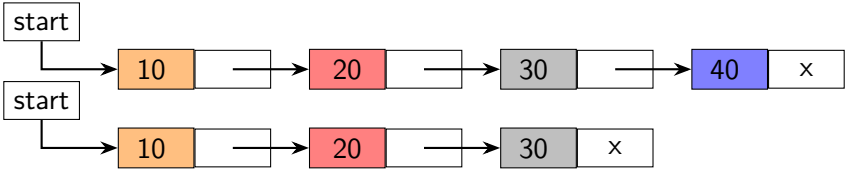
Pseudo Code: Delete first node

```
Step 1: IF START = NULL
            Write UNDERFLOW
            Go to Step 5
        [END OF IF]
Step 2: SET PTR = START
Step 3: SET START = START->NEXT
Step 4: FREE PTR
Step 5: EXIT
```

Singly Linked List Operations (Deletion of a Node) III

Case 2

Delete the last node.



Singly Linked List Operations (Deletion of a Node) IV

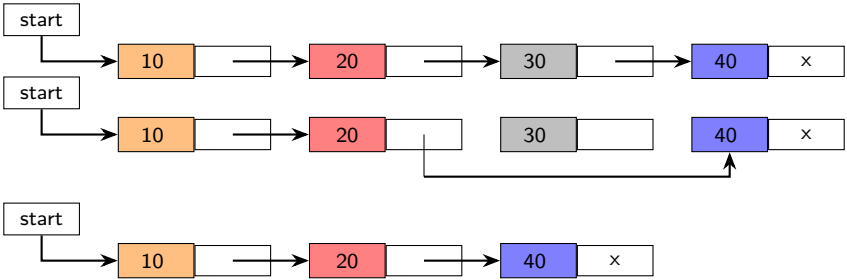
Pseudo Code: Delete last node

```
Step 1: IF START = NULL
            Write UNDERFLOW
            Go to Step 5
Step 2: SET PTR = START
Step 3: Repeat Steps 4 and 5 while PTR->NEXT != NULL
Step 4:     SET PREPTR = PTR
Step 5:     SET PTR = PTR->NEXT
        [END OF LOOP]
Step 6: SET PREPTR->NEXT = NULL
Step 7: FREE PTR
Step 8: EXIT
```

Singly Linked List Operations (Deletion of a Node) V

Case 3

Delete Node after the given node



Singly Linked List Operations (Deletion of a Node) VI

Pseudo Code: Delete Node after given node NUM

```
Step 1: IF START = NULL
            Write UNDERFLOW
            Go to Step 5
Step 2: SET PTR = START
Step 3: SET PREPTR = PTR
Step 4: Repeat Steps 5 and 6 while PREPTR -> DATA != NUM
Step 5:     SET PREPTR = PTR
Step 6:     SET PTR = PTR->NEXT
        [END OF LOOP]
Step 7: SET TEMP = PTR
Step 8: SET PREPTR->NEXT = PTR->NEXT
Step 9: FREE TEMP
Step 10: EXIT
```

Complexity of Singly Linked List Operation

Operations	Front/1st Node	k^{th} node	Back/ n^{th} Node
Find	$\Theta(1)$	$O(n)$	$\Theta(1)^*$
Insert Before	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Insert After	$\Theta(1)$	$\Theta(1)^\dagger$	$\Theta(1)^*$
Replace	$\Theta(1)$	$\Theta(1)^\dagger$	$\Theta(1)$
Erase	$\Theta(1)$	$O(n)$	$O(n)$
Next	$\Theta(1)$	$\Theta(1)^\dagger$	(n/a)‡
Previous	(n/a)‡	$\Theta(n)$	$\Theta(n)$

Table: Time complexity of Singly Linked List Operations

*Keeping a tail pointer

†Assuming you are currently at the kth node

‡Not Applicable

🤝 Reference & Q&A

Reference: Data Structures c Using 2nd Edition, Reema Thareja

👉 **Chapter 6**: 6.1, 6.2 (Covered topics only)

Thank You!

👏 Thank you for your time and attention. 👏

🗨 Any questions or clarifications. 💭